COLLEAGUE BRIEFING / 16 OCTOBER 2026

Cost of a private database

Proposal focus: two-party semi-honest computation. Historical ten-party analysis is retained on pages 2-4.

Current work: Garbled RAM. Proposed work: interactive 2PC with Circuit OPRAM. Public batch-path ordering and authorized output reordering cost no AND gates; this audit establishes no further numerical saving in the existing template.

The service we are evaluating

The target is **10,000 flat key-value reads per second**, with mean response below five seconds, for 2^{28} or 2^{32} records. Keys and values are each 32 bytes; both parties are in the same AWS availability zone. Merkle proof retrieval is outside this estimate.

Two-party proposal comparison

Conditional model result	2^{28} records	2^{32} records
Current: GRAM, assumed 100 ms state service	**$0.991 billion**	**$13.563 billion**
Proposed: interactive, NIC + SSD / I/O	**$70.44 million**	**$89.08 million**
Proposed: with assumed 200M AND/s/node	$83.28 million	$104.97 million

Annual costs include both parties. GRAM includes network, SSD capacity and attachment hosts; its CPU, full I/O and secure renewal remain unpriced. Interactive figures include modeled persistent I/O. Different unmeasured service assumptions prevent interpreting their ratio as a demonstrated saving.

What the routing audit establishes

Once duplicate-safe old paths are revealed, both parties can sort them and reorder their encoded request UIDs locally. Authorized outputs can be decoded and reordered locally. These public operations had no separate charge in the template, so the recalculated total is unchanged. The charged inverse routes belong to private pool sorting and extraction; reducing them requires a concrete replacement circuit.

Streamed Circuit OPRAM avoids future-program retention and exposes batch parallelism. The proposed row assumes batch 256 and one state, requiring complete service within **17.92 ms**. Neither that timing nor the CPU rate has been measured. Page 5 explains the sequential GRAM constraint; page 6 details the interactive estimate.

02 / HISTORICAL TEN-PARTY INTERACTIVE DESIGN

What the earlier estimate includes

Archived ten-party malicious model (up to four corruptions, security with abort): **$2.33B/$5.16B per year** at modeled mean responses of **4.77/3.97 seconds**, for the two database sizes. These are separate historical estimates, not the proposed two-party budget.

The working design places two cuckoo hash tables over binary Circuit OPRAM, a batched oblivious-memory construction. Every lookup probes both tables in parallel, checks the full key, and returns a value and membership result. Oblivious access hides the requested locations; even logical reads update the private placement state.

A central dispatcher forms batches before assigning them to independent replicas. Within each replica, several machines per party divide the state and computation. Each replica carries a complete, independently evolving ORAM state. This replication is a major reason the fleet is much larger than the raw database suggests.

Selected configuration	2^{28} records	2^{32} records
Raw key-value data	16 GiB	256 GiB
Queries per batch / map packing	128 / 128	512 / 256
Secure outputs per read	3.36 million	6.10 million
Sent data per read, per party	629.2 MB	1,085.8 MB
Persistent state per party, per replica	1.88 TiB	30.05 TiB
Machines per party, per replica	6	96
Independent replica groups	530	110
Total EC2 instances, all parties	**31,800**	**105,600**

MB is decimal; GiB and TiB are binary units. Traffic includes framing. Persistent state excludes scratch and preprocessing buffers. Map packing is the number of labels per recursive record; the interactive ORAM tree remains binary.

Why batching alone does not solve the cost

Batching exposes parallelism, but private comparisons, metadata conversion, payload movement, recursive-map updates and eviction still require substantial work. A private dot product can communicate one reduced output while performing many local multiplications. Counting only communicated outputs therefore misses a major cost driver.

At 10,000 reads/s, forming a 128-query batch adds only about **6.35 milliseconds** of mean waiting. The several-second response time comes mainly from computation and dependent exchanges. Moving work offline may reduce response latency, but its steady-state production must still be paid for.

The larger model has a lower selected latency because memory requirements force a larger parallel allocation. This is not evidence that increasing the database size improves performance. Both configurations are grid-search selections, not proven global optima.

03 / HISTORICAL TEN-PARTY COST DRIVERS

Local computation is the first bottleneck

For the smaller interactive reference, CPU demand exceeds network and memory demand. The model overlaps these resources, then adds dependent communication phases:

Per 128-query batch	Modeled time
CPU, including preprocessing	**3.45 s**
Network, including internal routing	2.22 s
RAM traffic	0.09 s
Dependent exchanges	1.29 s
Service time: 1.29 + max(3.45, 2.22, 0.09)	**4.74 s**

Preprocessing accounts for about 36% of CPU work and 24% of transmitted bytes; most remaining work is online. These are derived allocations that can change with scheduling. At 2^{32} records, RAM capacity also forces more machines.

A low-communication scan can still be expensive

A full-key scan compares the query with every record. An indexed two-dimensional scan uses private dot products to reduce communication but still processes the entire arrays locally. For the smaller database:

Per read, per party	Interactive OPRAM	Full-key scan	Indexed 2D scan
CPU proxy, billion operations	0.340	3,059	25.79
Sent data, decimal GB	0.629	6,075	0.041

The indexed scan uses about 6.5% of the reference traffic but **76 times its local arithmetic**. It remains a useful bandwidth-oriented benchmark if batched matrix kernels outperform the assumed CPU proxy.

Which assumptions move the answer most?

CPU and scheduling. For 2^{28} records, independently optimized faster- and slower-CPU scenarios cost $1.52 billion and $6.06 billion/year. Disabling resource overlap gives $3.96 billion/year. These are scenario results, not confidence limits.

Placement. The headline estimates assume same-availability-zone placement without marginal inter-party transfer fees. Cross-zone traffic is charged at $0.02 per transmitted GB across both endpoints in the retained price snapshot. Network placement can overwhelm compute savings.

Central assumptions include 2 billion weighted field operations/s per 64 vCPUs, 0.2 ms per exchange, 65% NIC efficiency and 70% operating utilization. None is a measured guarantee for this MPC service.

04 / HISTORICAL TEN-PARTY GARBLED-RAM MODEL

Preprocessing and retention dominate

Garbled RAM prepares an encoded computation before queries are evaluated. Few online rounds do not imply low total operating cost: at 10,000 reads/s, the prepared computation must be replenished continuously. In this model, **about 97.7% of transmitted bytes are preprocessing**.

The estimate assumes networking is the throughput bottleneck and sets garbled-stack and associated adaptor costs to zero under a proposed migration of a two-party optimization. This is a conditional multiparty construction, not an executed ten-party benchmark. CPU evaluation and a five-second response time have not been established.

The prepared horizon is a major budget choice

An epoch is the number of future queries prepared in advance; it is not a batch that waits to fill. Longer epochs amortize setup-related work over more queries but retain more garbled tables. The reference keeps two complete epochs, representing the current and next execution, plus encoded state.

Database / policy	Hours prepared per epoch	SSD per year	Total per year
2^{28}: reference and selected horizon	14.91	$2.14B	$10.33-18.52B
2^{32}: reference horizon	238.61	$44.44B	$55.07-65.69B
2^{32}: selected shorter horizon	14.91	$4.46B	$21.59-38.71B

B denotes billion USD; totals include network-capacity EC2 plus SSD for all parties. The selected horizon minimizes the upper endpoint within the tested grid. The range reflects party send/receive imbalance, not a complete uncertainty bound.

Shortening the larger case to about 15 hours substantially lowers the estimate. The reference SSD footprint is therefore a retention-policy consequence, not an unavoidable storage lower bound. Shorter epochs also increase recurring preparation per query, so the shortest horizon is not automatically cheapest.

What must be solved before this becomes a service estimate

The ledger does not fully price **malicious correlation generation, joint hidden-state initialization, secure epoch renewal, or the necessary representation conversions**. Accepting a continuing stream of external client queries also needs a service-security argument beyond the cited standalone protocol.

Securely carrying a live state into the next epoch requires more than re-encoding its bits. The model also leaves local evaluation speed, SSD throughput, machine allocation and cloud capacity unvalidated. **The sequential-throughput issue on page 5 also prevents treating these resource figures as a validated 10,000-read/s fleet.** Two-party coefficients cannot be substituted directly into this malicious model.

05 / TWO-PARTY SEMI-HONEST: GARBLED MERGE

Sequential evaluation drives replication

Both two-party models use two cuckoo tables of 2N slots, two probes and an eight-entry stash; each stored payload has 513 bits. Accounting uses 128-bit labels, FreeXOR and **32 bytes per AND** (half gates), with 16-byte merge joins counted separately. This is a backend substitution, not the artifact's native ThreeHalves gate size.

What PicoGRAM establishes [1, 2]

PicoGRAM describes its evaluator as mostly sequential. Its online-optimized variant **already avoids DDH/SIMD garbling**: a 64-bit RAM access takes 9.73 ms at 2^{16} words; 25 ms at 2^{24} and 41 ms at 2^{30} are paper extrapolations. These are not timings of garbled merge or of a two-probe, 64-byte lookup. Removing DDH therefore does not establish a faster evaluator; merge adds PRF work.

Recursive maps must return old positions before outer paths can be fetched; reinsertion and eviction modify shared state. The two cuckoo tables can run on two lanes, but adding cores does not give linear speedup for successive accesses to either table. SIMD and inter-request overlap need an implemented schedule.

Required copies S = ceil(10,000 x L / 0.70), where L is the complete state service time in seconds, including maintenance before the next query. 128 copies require L <= 8.96 ms; 500 require L <= 35 ms.

Assumed service per KV	Required states	Annual 2^{28}	Annual 2^{32}
100 ms	1,429	$0.991B	$13.563B
500 ms	7,143	$4.307B	$65.839B
1 second	14,286	$8.378B	$130.621B

Conditional **network EC2 + SSD capacity and attachment hosts**, both parties. Each row reoptimizes the prepared horizon at a fixed, unmeasured L. Two complete epochs, literal labels and separate one-time programs per state are retained. CPU, full SSD I/O, temporary space, initialization and secure renewal are additional; these are not GRAM cost lower bounds.

Communication savings do not establish runtime

At horizon T = 2N and map packing 8, the source-derived model produces **61.47/80.38 MB of tables per lookup**, including merge and output bindings. Live labels occupy 49.39/799.91 TB per party per state. The literal artifact's full-horizon merge computation is impractical at these sizes; small-window resets and handoff need implementation. Aggregate hash throughput cannot replace the service-time constraint.

For scale, a hypothetical 64-call reset window gives 30.40M/41.28M evaluator merge hashes per lookup at T = 2N. At 40 cycles/hash and 2.2 GHz, that is 0.55/0.75 seconds of scalar merge work alone. Two table lanes can overlap; SIMD may help. This is a work sensitivity with unimplemented reset/handoff and assumed visit statistics, not a measured latency or circuit-depth bound.

06 / TWO-PARTY SEMI-HONEST: STREAMED BATCHES

Trade retained programs for interaction

The evaluator reveals duplicate-safe old fetch positions or independent dummy paths to the garbler, which streams the next circuit. Both parties can sort these public paths and reorder encoded request UIDs locally: **no garbled Waksman network is needed**. Fresh replacement positions remain hidden [3].

Public ordering is free; private pool extraction is unresolved

Authorized data outputs and selected old recursive-map positions may be decoded and restored to request order locally. A public permutation also reorders encoded outputs without AND gates. These operations had no separate charge in the template. Its inverse-route allowances instead accompany hidden pool sorting, state alignment and result extraction. **They are not proved necessary**, but removing them requires a complete alternative extraction/compaction circuit, including conflicts and internal state updates.

Explicit Boolean template, batch 256

Per lookup / annual resource model	2^{28} records	2^{32} records
AND gates per lookup	12.921 million	16.008 million
Garbled table bytes per lookup	413.5 MB	512.3 MB
Live label state per party, one copy	63.095 TB	1.032 PB
NIC + SSD capacity / modeled I/O, per year	**$70.44 million**	**$89.08 million**

Both parties, one state, decimal units. Recalculation preserves these totals: the verified change is 0 ANDs. At an assumed effective 200M AND/s/node, CPU provisioning raises totals to $83.28/104.97M. Pool composition, recursive updates and eviction remain planning allowances; this is not an executed trace or a proved finite upper bound.

Fresh positions are XORs of private party inputs; FreeXOR adds no ANDs. Labels and OT are charged separately: fresh-position input traffic is 17.92/23.04 KB/lookup. Joint hidden initialization remains unpriced.

Batch latency must justify the state count

$S = \text{ceil}(10{,}000 \times L_{batch} / (0.70 \times B))$. One state at B = 256 requires service <= 17.92 ms. At B = 1,024 the limit is 71.68 ms: a 50 ms batch needs one state; 500 ms needs seven. CPU throughput alone does not establish these latencies.

At B = 1,024, the unchanged template gives 541.2/665.7 MB/lookup and $91.68/114.62M/year. Tables are streamed and discarded; live state remains. Compact XOR shares could reduce the 16-byte-per-bit state representation, with conversion/OT costs still to price.

07 / RECOMMENDED NEXT STEPS AND SOURCES

Measure service time before choosing a fleet

Focus the proposal on **two-party semi-honest computation**: current Garbled RAM versus proposed interactive Circuit OPRAM. Its opportunity is batch parallelism and less retained garbled code. Public request/output ordering is free, but does not by itself eliminate private pool processing.

1. Measure a complete state cycle

For garbled merge, measure both table probes, recursive maps, PRFs, label/table I/O, eviction and the proposed reset/renewal mechanism. Report state service time and active core-seconds separately. For batch OPRAM, measure full batch depth, dependent exchanges and worker scaling, including duplicate queries, misses and map-block conflicts.

2. Optimize, then enforce the throughput constraint

Replace the generic pool inverse-route allowances with a concrete extraction and compaction design before subtracting them. Preserve hidden resident UIDs, destinations and real/dummy status. Then tune packing, batch size and the flat base; derive copies from measured service time at 70% utilization before pricing workers and storage.

3. Calibrate cost and security together

Validate joint hidden initialization, duplicate-safe path revelation, position refresh, state transitions and lifetime overflow bounds. Measure CPU, network and storage I/O on the intended topology. Streaming retains live state; short-epoch renewal and compact state conversion need complete accounting. Pages 2-4 preserve the earlier ten-party study as historical context.

Price basis. Two-party figures use AWS US East Linux/shared On-Demand prices refreshed 14 October 2026 (feed version 20261012193925): c6in.32xlarge $7.2576/hour, 200 Gbps NIC; gp3 $0.96/GiB-year. Network planning uses 65% efficiency, 70% distributed efficiency and 70% utilization, with both roles charged. These factors are not measured protocol throughput.

Scope. Flat read-only key-value service with private client-supplied hash indices; no Merkle proofs. Setup, a client gateway, backups, failover reserve, support, taxes and engineering are unpriced. Coverage differs across tables; every quoted range is conditional, not a confidence interval or complete budget.

Evidence and reproduction

[1] Liu et al., *Improved Garbled RAM via Garbled Merge*, ePrint 2026/761, final revision 9 August 2026; artifact v2. Byte/hash measurements do not establish end-to-end service throughput.

[2] *PicoGRAM*, ePrint 2025/1479, Table 2, Section 7, Appendices D.2 and E.3. Communication-, end-to-end- and online-optimized variants are distinct; their best numbers cannot be combined.

[3] Chan and Shi, *Circuit OPRAM*, ePrint 2016/1084. The two-party Boolean cost template is this study's proposed compilation, not a benchmark reported by that paper.

[4] Historical ten-party figures: retained in the full report. Current two-party derivations and routing audit: *mpc_cost/data/two_party_notes.txt*, *two_party_accuracy_review.txt* and *two_party_serial_notes.txt*; results: *two_party_comparison.json* and *two_party_serial_comparison.json* under *mpc_cost/results/*. Commands: *mpc_cost/README.txt*.